RNNs and LSTMs from Scratch - HW3

Total Points: 100

In this assignment, you will implement RNNs and LSTMs from scratch using PyTorch. You will complete the missing code sections marked with `#TODO` comments. Each section is worth the indicated points.

Learning Objectives

1. Understand the architecture of RNNs and LSTMs
2. Implement forward passes for recurrent architectures
3. Train and evaluate sequence models
4. Apply RNNs/LSTMs to text classification and language modeling tasks

Grading Breakdown

- Part 1: RNN Cell Implementation (20 points)
- Part 2: RNN with Embedding Forward Pass (15 points)
- Part 3: Stacked RNN Implementation (20 points)
- Part 4: LSTM Cell Implementation (25 points)
- Part 5: Language Model Implementation (20 points)

Instructions: Replace all `#TODO` sections with your implementation. Make sure to read the hints carefully!

RNNs are recurrent neural networks, a class of neural networks that is well-suited for processing sequential data. They are widely used in various applications such as natural language processing, time series prediction, and speech recognition.

LSTMs, or Long Short-Term Memory networks, are a type of RNN that addresses the vanishing gradient problem encountered in traditional RNNs. They are capable of learning long-term dependencies in sequential data by using special units called memory cells.

In this assignment, we will implement RNNs and LSTMs from scratch using PyTorch (autograd takes care of the backward pass). We will cover the following topics:

1. Understanding the architecture of RNNs and LSTMs
2. Implementing the forward pass
3. Training the models
4. Evaluating the models

Let's get started!

1. Understanding the Architecture

RNN Architecture

An RNN processes input sequences one element at a time while maintaining a hidden state that captures information about previous elements in the sequence. The basic equations for an RNN are:

- Hidden state update:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- Output:

$$y_t = W_{hy}h_t + b_y$$

Where:

- h_t is the hidden state at time step t
- x_t is the input at time step t
- W_{hh}, W_{xh}, and W_{hy} are weight matrices
- b_h and b_y are bias vectors

LSTM Architecture

An LSTM consists of several components: input gate, forget gate, output gate, and cell state. The equations for an LSTM are:

- Forget gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

- Input gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

- Candidate cell state:

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

- Cell state update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

- Output gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

- Hidden state:

$$h_t = o_t \odot \tanh(C_t)$$

Where:

- $[h_{t-1}, x_t]$ is the concatenation of the previous hidden state and the current input
- σ is the sigmoid function and $\odot$ is element-wise multiplication
- C_t is the cell state (long-term memory)

In Part 4 you will implement the same equations in the equivalent form with separate input-to-hidden and hidden-to-hidden weights ($W_{ih}x_t + W_{hh}h_{t-1}$ instead of $W[h_{t-1}, x_t]$).

Setup and Data Loading

The following cells handle imports and data loading. **You don't need to modify these cells.**

In [1]:
```
# --- Imports ---
import torch                          # core tensor library + autograd (computes
import torch.nn as nn                 # building blocks: nn.Module, nn.Linear, n
import torch.nn.functional as F       # stateless functions: F.linear, activatio
from torchvision import models
from typing import List, Tuple        # type hints only (no runtime effect)
import math                           # math.sqrt used for weight initialization
import numpy as np

import datasets                       # Hugging Face `datasets` library, used to
```

In [2]:
```
# AG News: news headlines + short descriptions, each labeled with one of 4 t
# (0=World, 1=Sports, 2=Business, 3=Sci/Tech).
# We take only the first 10% of the training split (~12k examples) so traini
# and the full test split (7.6k examples) for validation.
# Each example is a dict like {"text": "...", "label": 2}.
data = datasets.load_dataset("fancyzhx/ag_news", split="train[:10%]")
data_val = datasets.load_dataset("fancyzhx/ag_news", split="test")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
```

In [3]:
```
import re
from collections import Counter

# ------------------------------------------------------------------------
# 1) Tokenization: turn a raw string into a list of word tokens.
# ------------------------------------------------------------------------
def tokenize_basic(text: str):
    # lowercase + keep alnum + simple contractions
    # The regex matches runs of letters/digits, optionally followed by an ap
    # and more letters/digits, so "Don't stop!" -> ["don't", "stop"].
```

```
    # Punctuation and other symbols are simply dropped.
    return re.findall(r"[a-z0-9]+(?:'[a-z0-9]+)?", text.lower())

# Build vocab from TRAIN TEXT ONLY
# (using validation text here would leak information about the test set into
train_texts = list(data["text"])

# ------------------------------------------------------------------------
# 2) Vocabulary: map every known word to an integer id.
# ------------------------------------------------------------------------
def build_vocab(texts, max_vocab_size=30000, min_freq=2):
    # Count how many times each token appears across all training texts
    counter = Counter()
    for t in texts:
        counter.update(tokenize_basic(t))
    # keep words meeting min_freq, capped by max_vocab_size
    # most_common() is sorted by frequency, so we keep the most frequent wor
    # words seen only once are mostly typos/rare names and would just add no
    most_common = [w for (w, c) in counter.most_common() if c >= min_freq][:
    # Two special tokens go first, so they get fixed ids:
    #   <pad> = 0 : filler used to make all sequences the same length
    #   <unk> = 1 : stands for any word that is not in the vocabulary
    vocab = ["<pad>", "<unk>"] + most_common
    # word -> id lookup table (the id is just the position in `vocab`)
    word2idx = {w: i for i, w in enumerate(vocab)}
    return vocab, word2idx

vocab, word2idx = build_vocab(train_texts, max_vocab_size=30000, min_freq=2)

# ------------------------------------------------------------------------
# 3) Tokenizer: converts text <-> list of ids using the vocabulary above.
# ------------------------------------------------------------------------
class Tokenizer:
    def __init__(self, vocab, word2idx):
        self.vocab = vocab
        self.word2idx = word2idx
        # Reverse lookup (id -> word), needed for decoding
        self.idx2word = {idx: word for word, idx in word2idx.items()}
        self.pad_token = "<pad>"
        self.unk_token = "<unk>"
        self.pad_idx = self.word2idx[self.pad_token]

    def encode(self, text: str):
        # text -> tokens -> ids; unknown words map to the <unk> id
        toks = tokenize_basic(text)
        return [self.word2idx.get(w, self.word2idx[self.unk_token]) for w in

    def decode(self, indices):
        # ids -> words joined by spaces (useful for inspecting / debugging)
        return " ".join([self.idx2word.get(i, self.unk_token) for i in indic

tokenizer = Tokenizer(vocab, word2idx)


# ------------------------------------------------------------------------
# 4) PyTorch Dataset for classification.
```

```
#    Each item is (token_ids, label, length), where token_ids always has
#    exactly `max_length` entries so items can be stacked into a batch.
# ----------------------------------------------------------------------------
class Dataset(torch.utils.data.Dataset):
    def __init__(self, data, tokenizer: Tokenizer, max_length: int = 50):
        self.data = data
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.pad_idx = tokenizer.pad_idx

    def __len__(self):
        # DataLoader uses this to know how many examples there are
        return len(self.data)

    def __getitem__(self, idx):
        text = self.data[idx]["text"]
        label = self.data[idx]["label"]
        ids = self.tokenizer.encode(text)
        # Number of REAL tokens (before padding). The models use it to find
        # last real token, so they ignore the padding at the end.
        length = min(len(ids), self.max_length)
        # Truncate texts that are too long...
        ids = ids[:self.max_length]
        # ...and right-pad texts that are too short with <pad> ids
        if len(ids) < self.max_length:
            ids = ids + [self.pad_idx] * (self.max_length - len(ids))
        return torch.tensor(ids, dtype=torch.long), torch.tensor(label, dtyp
```

In [4]:
```python
class LanguageModelDataset(torch.utils.data.Dataset):
    """Dataset for language modeling tasks (no labels needed)"""
    # Same idea as `Dataset` above (encode -> truncate -> pad), but a langua
    # learns to predict the next word from the text itself, so the topic lab
    def __init__(self, data, tokenizer: Tokenizer, max_length: int = 50):
        self.data = data
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.pad_idx = tokenizer.pad_idx

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        text = self.data[idx]["text"]
        # Skip empty texts
        # (an empty text would give zero tokens; replace it with a dummy sen
        if not text.strip():
            text = "empty text"
        ids = self.tokenizer.encode(text)
        length = min(len(ids), self.max_length)
        ids = ids[:self.max_length]
        if len(ids) < self.max_length:
            ids = ids + [self.pad_idx] * (self.max_length - len(ids))
        # For language modeling, we return dummy label and length
        # (the label is a dummy 0; the 3-tuple keeps the same format as `Dat
        #  so the same DataLoader loop `for inputs, _, _ in ...` works)
        return torch.tensor(ids, dtype=torch.long), torch.tensor(0, dtype=to
```

Part 1: RNN Cell Implementation (20 points)

Implement the RNN cell from scratch. The RNN cell computes:

$$h_t = \tanh(W_{ih} \cdot x_t + b_{ih} + W_{hh} \cdot h_{t-1} + b_{hh})$$

Where:

- x_t is the input at time step t
- h_{t-1} is the previous hidden state
- W_{ih}, W_{hh} are weight matrices
- b_{ih}, b_{hh} are bias vectors

In [5]:
```python
# A single RNN step: takes the current input x_t and the previous hidden sta
# and returns the new hidden state h_t. The full RNN (Part 2) calls this cel
# time step, reusing the SAME weights at every step.
class RNN_cell(nn.Module):
    def __init__(self, input_size: int, hidden_size: int):
        super().__init__()
        self.input_size = input_size     # size of x_t (e.g. the embedding d
        self.hidden_size = hidden_size   # size of h_t (the RNN's "memory" v

        # TODO: Define the weights and biases (part of Part 1)
```

```
        # Hint 1: Wrap each tensor in nn.Parameter so it is registered with
        #         initialized by init_weights() below, and updated by the op
        # Hint 2: F.linear(x, W, b) computes x @ W.T + b, so W has shape (ou
        # Hint 3: W_ih: (hidden_size, input_size), W_hh: (hidden_size, hidde
        # Hint 4: Create an uninitialized tensor with torch.Tensor(rows, col
        #         (LSTMCell in Part 4 uses the same pattern with 4 * hidden_
        self.W_ih = nn.Parameter(torch.Tensor(hidden_size, input_size))
        self.W_hh = nn.Parameter(torch.Tensor(hidden_size, hidden_size))
        self.b_ih = nn.Parameter(torch.Tensor(hidden_size))
        self.b_hh = nn.Parameter(torch.Tensor(hidden_size))

        self.init_weights()

    def init_weights(self):
        # Fill every parameter with small random values from U(-1/sqrt(H), 1
        # This is the same scheme PyTorch's built-in nn.RNN uses; small weig
        # tanh out of its flat (saturated) region at the start of training.
        stdv = 1.0 / math.sqrt(self.hidden_size)
        for weight in self.parameters():
            weight.data.uniform_(-stdv, stdv)

    def forward(self, x: torch.Tensor, h_prev: torch.Tensor) -> torch.Tensor
        # Shapes: x is (B, input_size), h_prev is (B, hidden_size) -> output
        # TODO: Implement the RNN cell forward pass (20 points)
        # Hint 1: Use F.linear(input, weight, bias) to compute linear transf
        # Hint 2: Apply F.linear to x with W_ih and b_ih
        # Hint 3: Apply F.linear to h_prev with W_hh and b_hh
        # Hint 4: Add the two results together
        # Hint 5: Apply tanh activation: torch.tanh()
        # Your code should be a single return statement combining all these

        return torch.tanh(F.linear(x, self.W_ih, self.b_ih) + F.linear(h_pre
```

Part 2: RNN with Embedding Forward Pass (15 points)

Complete the forward pass of the RNN model with embedding layer. The model should:

1. Embed the input tokens
2. Process each time step through the RNN cell
3. Extract the hidden state at the last valid token position
4. Pass it through a fully connected layer for classification

In [6]:
```
# Full classifier: token ids -> embeddings -> RNN over time -> last hidden s
class RNN_with_embedding(nn.Module):
    def __init__(self, max_token_len: int, vocab_size: int, embedding_dim: i
        super().__init__()
        # Embedding layer (padding_idx freezes PAD embedding to zeros)
        # It is a lookup table: row i is the learned vector for word id i.
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx
        # One RNN cell, applied repeatedly at every time step
```

```
        self.rnn_cell = RNN_cell(embedding_dim, hidden_size)
        # Final linear layer: hidden state -> one score (logit) per class
        self.fc = nn.Linear(hidden_size, output_size)
        self.max_token_len = max_token_len
        self.hidden_size = hidden_size

    def forward(self, x: torch.Tensor, lengths: torch.Tensor) -> torch.Tenso
        # x: (B, L) token ids; lengths: (B,) number of real (non-pad) tokens
        # B = batch size, L = sequence length, E = embedding dim, H = hidden
        emb = self.embedding(x)  # (B, L, E)
        B, L, E = emb.size()

        # TODO: Initialize hidden state (3 points)
        # Hint: Create a zero tensor of shape (B, self.hidden_size) on the s
        # Use: torch.zeros(..., device=emb.device)
        h = torch.zeros(B, self.hidden_size, device=emb.device)

        hs = []  # will collect the hidden state from every time step

        # TODO: Unroll RNN through time (7 points)
        # Hint 1: Loop through each time step: for t in range(L)
        # Hint 2: Get input at time t: emb[:, t] which has shape (B, E)
        # Hint 3: Pass through RNN cell: h = self.rnn_cell(input_t, h)
        # Hint 4: Store hidden state: hs.append(h.unsqueeze(1))
        #TODO: Implement time loop
        for t in range (0, L):
            input_t = emb[:, t]
            h = self.rnn_cell(input_t, h)
            hs.append(h.unsqueeze(1))
        # Stack the per-step hidden states along the time axis
        Hseq = torch.cat(hs, dim=1)  # (B, L, H)

        # Why not just use the hidden state at the final position L-1?
        # Because short texts are padded: for them the final positions are <
        # so we pick the hidden state at each example's own last REAL token.
        # TODO: Extract last valid hidden state (5 points)
        # Hint 1: Get indices of last valid tokens: idx = (lengths - 1).clam
        # Hint 2: Create batch indices: batch_indices = torch.arange(B, devi
        # Hint 3: Index into Hseq: last_h = Hseq[batch_indices, idx]
        idx = (lengths-1).clamp_min(0)
        batch_indices = torch.arange(B, device=emb.device)
        last_h = Hseq[batch_indices, idx]

        # (B, H) -> (B, output_size) logits; CrossEntropyLoss applies softma
        return self.fc(last_h)
```

In [7]:
```
# Model hyperparameters
max_token_len = 128          # every text is truncated/padded to 128 tokens
vocab_size = len(vocab)      # number of rows in the embedding table
embedding_dim = 50           # size of each word vector
hidden_size = 100            # size of the RNN hidden state
output_size = 4  # 4 classes in AG News
pad_idx = tokenizer.pad_idx

# Instantiate model
model = RNN_with_embedding(max_token_len, vocab_size, embedding_dim, hidden_
```

```
print(model)

# Create data loaders
# A DataLoader groups dataset items into batches of 32. Each batch is a tupl
# (inputs (32, 128), labels (32,), lengths (32,)). Training data is shuffled
# epoch so the model does not see examples in the same order; validation is
dataset = Dataset(data, tokenizer, max_length=max_token_len)
dataset_val = Dataset(data_val, tokenizer, max_length=max_token_len)
dataloader = torch.utils.data.DataLoader(dataset, batch_size=32, shuffle=Tru
dataloader_val = torch.utils.data.DataLoader(dataset_val, batch_size=32, shu
```

```
RNN_with_embedding(
  (embedding): Embedding(15994, 50, padding_idx=0)
  (rnn_cell): RNN_cell()
  (fc): Linear(in_features=100, out_features=4, bias=True)
)
```

Training Functions

The following training and validation functions are provided. **You don't need to modify these.**

In [8]:

```
from tqdm import tqdm  # progress bars for loops

def validate(model, dataloader_val, criterion, device):
    # Evaluate the model on the validation set: average loss and accuracy (%
    model.eval()  # switch to evaluation mode (matters for dropout/batchnorm
    total_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():  # no gradients needed -> faster and uses less mem
        for inputs, labels, lengths in tqdm(dataloader_val):
            inputs, labels, lengths = inputs.to(device), labels.to(device),
            outputs = model(inputs, lengths)        # (B, num_classes) logit
            loss = criterion(outputs, labels)
            total_loss += loss.item()
            preds = outputs.argmax(dim=1)           # predicted class = high
            total += labels.size(0)
            correct += (preds == labels).sum().item()
    return total_loss / len(dataloader_val), 100.0 * correct / total

def train(model, dataloader, dataloader_val, num_epochs: int = 5, learning_r
    # Use the GPU if there is one, otherwise the CPU
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)
    # CrossEntropyLoss = softmax + negative log-likelihood of the correct cl
    criterion = nn.CrossEntropyLoss()
    # Adam updates every parameter registered in the model (incl. our nn.Par
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    history = {"train_loss": [], "val_loss": [], "val_accuracy": []}

    for epoch in range(num_epochs):  # one epoch = one pass over the whole t
        model.train()
        total_loss = 0.0
        for inputs, labels, lengths in tqdm(dataloader):
            inputs, labels, lengths = inputs.to(device), labels.to(device),
```

```
            # The standard 5-step PyTorch training step:
            optimizer.zero_grad()               # 1. clear gradients from the
            outputs = model(inputs, lengths)    # 2. forward pass
            loss = criterion(outputs, labels)   # 3. compute loss
            loss.backward()                     # 4. backprop: autograd compu
            optimizer.step()                    # 5. update the weights
            total_loss += loss.item()

        # After each epoch: record average training loss and evaluate on val
        avg_train_loss = total_loss / len(dataloader)
        val_loss, val_acc = validate(model, dataloader_val, criterion, devic
        history["train_loss"].append(avg_train_loss)
        history["val_loss"].append(val_loss)
        history["val_accuracy"].append(val_acc)
        print(f"Epoch [{epoch+1}/{num_epochs}] Train: {avg_train_loss:.4f} V
    return history

# Train the model (uncomment after implementing the above TODOs)
history = train(model, dataloader, dataloader_val, num_epochs=5, learning_ra
```

```
100%|██████████| 375/375 [00:16<00:00, 22.32it/s]
100%|██████████| 238/238 [00:03<00:00, 71.38it/s]
Epoch [1/5] Train: 1.2772 Val: 1.2161 Acc: 43.57%
100%|██████████| 375/375 [00:15<00:00, 23.74it/s]
100%|██████████| 238/238 [00:03<00:00, 79.30it/s]
Epoch [2/5] Train: 1.0011 Val: 1.0497 Acc: 53.16%
100%|██████████| 375/375 [00:16<00:00, 22.74it/s]
100%|██████████| 238/238 [00:02<00:00, 87.49it/s]
Epoch [3/5] Train: 0.7993 Val: 0.9297 Acc: 62.28%
100%|██████████| 375/375 [00:14<00:00, 25.67it/s]
100%|██████████| 238/238 [00:02<00:00, 82.14it/s]
Epoch [4/5] Train: 0.6353 Val: 0.9979 Acc: 61.00%
100%|██████████| 375/375 [00:14<00:00, 25.41it/s]
100%|██████████| 238/238 [00:02<00:00, 84.48it/s]
Epoch [5/5] Train: 0.5436 Val: 0.8526 Acc: 68.63%
```

Part 3: Stacked/Multi-layer RNN (20 points)

Implement a stacked RNN architecture where the output of one RNN layer becomes the input to the next layer. This allows the model to learn more complex hierarchical representations.

Note: the 3-layer RNN can train unstably: its validation accuracy may jump around between epochs or drop in the last epochs. This does not by itself mean your implementation is wrong. If you want to dig into why, see the **Bonus** at the end of the notebook.

In [9]:
```
# Stacked (deep) RNN: several RNN cells on top of each other. At every time
# layer 0 reads the word embedding, and each higher layer reads the hidden s
# of the layer below it. Only the top layer's hidden states are used for the
class RNNWithStackedLayers(nn.Module):
```

```
    def __init__(self, max_token_len: int, vocab_size: int, embedding_dim: i
                 hidden_size: int, output_size: int, pad_idx: int, num_layer
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx

        # TODO: Create a list of RNN cells for each layer (8 points)
        # Hint 1: Use nn.ModuleList to store multiple RNN cells
        # Hint 2: First layer takes embedding_dim as input_size
        # Hint 3: Subsequent layers take hidden_size as input_size
        # Hint 4: All layers have hidden_size as output size
        # Example structure: [RNN_cell(embedding_dim, hidden_size), RNN_cell
        # (Note: a plain Python list would NOT register the cells' parameter
        #  model, so the optimizer would never train them. nn.ModuleList doe
        self.rnn_cells = nn.ModuleList(
            [RNN_cell(embedding_dim if i == 0 else hidden_size, hidden_size)

        self.fc = nn.Linear(hidden_size, output_size)
        self.max_token_len = max_token_len
        self.hidden_size = hidden_size
        self.num_layers = num_layers

    def forward(self, x: torch.Tensor, lengths: torch.Tensor) -> torch.Tenso
        emb = self.embedding(x)  # (B, L, E)
        B, L, E = emb.size()

        # Each layer keeps its own hidden state (its own "memory")
        # TODO: Initialize hidden states for all layers (4 points)
        # Hint: Create a list of num_layers hidden states, each of shape (B,
        # Use list comprehension: [torch.zeros(...) for _ in range(self.num_
        h = [torch.zeros(B, self.hidden_size, device=emb.device) for _ in ra

        # TODO: Initialize list to store hidden states for each layer (1 poi
        # Hint: Create a list of empty lists: [[] for _ in range(self.num_la
        hs = [[] for _ in range(self.num_layers)]

        # TODO: Unroll through time with stacked layers (7 points)
        # Hint 1: Outer loop: for t in range(L)
        # Hint 2: Set input_t = emb[:, t] initially
        # Hint 3: Inner loop: for layer in range(self.num_layers)
        # Hint 4: Update hidden state: h[layer] = self.rnn_cells[layer](inpu
        # Hint 5: Store hidden state: hs[layer].append(h[layer].unsqueeze(1)
        # Hint 6: Update input_t for next layer: input_t = h[layer]
        #TODO: Implement nested loops
        for t in range(L):
            input_t = emb[:, t]
            for layer in range(self.num_layers):
                h[layer] = self.rnn_cells[layer](input_t, h[layer])
                hs[layer].append(h[layer].unsqueeze(1))
                input_t = h[layer]


        # Use last layer's hidden states
        Hseq = hs[-1]
        Hseq = torch.cat(Hseq, dim=1)  # (B, L, H)

        # Extract last valid hidden state
```

```
        # (same trick as Part 2: for each example, pick the step of its last
        idx = (lengths - 1).clamp_min(0)
        batch_indices = torch.arange(B, device=emb.device)
        last_h = Hseq[batch_indices, idx]

        return self.fc(last_h)

# Instantiate stacked RNN
model_stacked = RNNWithStackedLayers(max_token_len, vocab_size, embedding_di
                                     hidden_size, output_size, pad_idx, num
print(model_stacked)

# Train the model (uncomment after implementing the above TODOs)
history_stacked = train(model_stacked, dataloader, dataloader_val, num_epoch
```

```
RNNWithStackedLayers(
  (embedding): Embedding(15994, 50, padding_idx=0)
  (rnn_cells): ModuleList(
    (0-2): 3 x RNN_cell()
  )
  (fc): Linear(in_features=100, out_features=4, bias=True)
)
100%|██████████| 375/375 [00:33<00:00, 11.15it/s]
100%|██████████| 238/238 [00:06<00:00, 35.70it/s]
Epoch [1/5] Train: 1.2483 Val: 1.2727 Acc: 40.34%
100%|██████████| 375/375 [00:34<00:00, 10.93it/s]
100%|██████████| 238/238 [00:06<00:00, 36.83it/s]
Epoch [2/5] Train: 1.0291 Val: 1.1110 Acc: 49.26%
100%|██████████| 375/375 [00:33<00:00, 11.13it/s]
100%|██████████| 238/238 [00:06<00:00, 35.81it/s]
Epoch [3/5] Train: 0.8971 Val: 1.0489 Acc: 52.41%
100%|██████████| 375/375 [00:34<00:00, 11.01it/s]
100%|██████████| 238/238 [00:06<00:00, 36.47it/s]
Epoch [4/5] Train: 0.8008 Val: 1.2003 Acc: 45.20%
100%|██████████| 375/375 [00:33<00:00, 11.08it/s]
100%|██████████| 238/238 [00:06<00:00, 35.97it/s]
Epoch [5/5] Train: 0.9519 Val: 1.0479 Acc: 51.72%
```

Part 4: LSTM Cell Implementation (25 points)

Implement an LSTM cell from scratch. LSTMs use gates to control information flow:

- **Input gate** (i_t): Controls what new information to add
- **Forget gate** (f_t): Controls what information to discard
- **Output gate** (o_t): Controls what to output
- **Cell state** (c_t): Long-term memory

The LSTM equations are:

$$gates = W_{ih} \cdot x_t + b_{ih} + W_{hh} \cdot h_{t-1} + b_{hh} \tag{1}$$
$$i_t, f_t, g_t, o_t = \text{split}(gates, 4) \tag{2}$$
$$i_t = \sigma(i_t), \quad f_t = \sigma(f_t), \quad o_t = \sigma(o_t) \tag{3}$$
$$g_t = \tanh(g_t) \tag{4}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \tag{5}$$
$$h_t = o_t \odot \tanh(c_t) \tag{6}$$

In [10]:
```python
# A single LSTM step. Besides the hidden state h, an LSTM carries a cell sta
# ("long-term memory"). Gates (values between 0 and 1) decide what to forget
# what new information to write into c, and how much of c to expose as h.
class LSTMCell(nn.Module):
    def __init__(self, input_size: int, hidden_size: int):
        super().__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size

        # Weights for all 4 gates combined (input, forget, cell, output)
        # Instead of 4 separate weight matrices, they are stacked into one m
        # 4 * hidden_size rows, so all gates are computed with one matrix mu
        # then split into 4 chunks of size hidden_size.
        self.W_ih = nn.Parameter(torch.Tensor(4 * hidden_size, input_size))
        self.W_hh = nn.Parameter(torch.Tensor(4 * hidden_size, hidden_size))
        self.b_ih = nn.Parameter(torch.Tensor(4 * hidden_size))
        self.b_hh = nn.Parameter(torch.Tensor(4 * hidden_size))
        self.init_weights()

    def init_weights(self):
        # Same uniform initialization as RNN_cell
        stdv = 1.0 / math.sqrt(self.hidden_size)
        for weight in self.parameters():
            weight.data.uniform_(-stdv, stdv)

    def forward(self, x: torch.Tensor, h_prev: torch.Tensor, c_prev: torch.T
        # Shapes: x (B, input_size), h_prev and c_prev (B, hidden_size)
        # Returns the new (h, c), both (B, hidden_size)
        # TODO: Compute combined gates (5 points)
        # Hint: gates = F.linear(x, self.W_ih, self.b_ih) + F.linear(h_prev,
        gates = F.linear(x, self.W_ih, self.b_ih) + F.linear(h_prev, self.W_

        # TODO: Split gates into input, forget, cell, output gates (4 points
        # Hint: Use gates.chunk(4, 1) to split into 4 equal parts along dime
        # This returns a tuple: (i, f, g, o)
        i, f, g, o = gates.chunk(4, 1)

        # Sigmoid squashes to (0, 1) -> acts like a soft on/off switch for t
        # Tanh squashes to (-1, 1) -> used for the candidate values g writte
        # TODO: Apply activations to gates (8 points - 2 each)
        # Hint 1: Apply sigmoid to i: i = torch.sigmoid(i)
        # Hint 2: Apply sigmoid to f: f = torch.sigmoid(f)
        # Hint 3: Apply tanh to g: g = torch.tanh(g)
        # Hint 4: Apply sigmoid to o: o = torch.sigmoid(o)
        i = torch.sigmoid(i)
        f = torch.sigmoid(f)
```

```
        g = torch.tanh(g)
        o = torch.sigmoid(o)

        # TODO: Update cell state (4 points)
        # Hint: c = f * c_prev + i * g
        # The forget gate controls what to keep from c_prev
        # The input gate controls what to add from g
        # (The update is additive, so gradients can flow back through c acro
        #  steps without vanishing as quickly as in a plain RNN.)
        c = f * c_prev + i * g

        # TODO: Compute hidden state (4 points)
        # Hint: h = o * torch.tanh(c)
        # The output gate controls what to output from the cell state
        h = o * torch.tanh(c)

        return h, c
```

In [11]:

```
# Same classifier as Part 2, but the RNN cell is replaced by the LSTM cell.
# This part is already complete: it only needs your LSTMCell to work.
class LSTMWithEmbedding(nn.Module):
    def __init__(self, max_token_len: int, vocab_size: int, embedding_dim: i
                 hidden_size: int, output_size: int, pad_idx: int):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx
        self.lstm_cell = LSTMCell(embedding_dim, hidden_size)
        self.fc = nn.Linear(hidden_size, output_size)
        self.max_token_len = max_token_len
        self.hidden_size = hidden_size

    def forward(self, x: torch.Tensor, lengths: torch.Tensor) -> torch.Tenso
        emb = self.embedding(x)  # (B, L, E)
        B, L, E = emb.size()

        # Initialize hidden and cell states
        # (both start as zeros: "no memory" before reading the first word)
        h = torch.zeros(B, self.hidden_size, device=emb.device)
        c = torch.zeros(B, self.hidden_size, device=emb.device)
        hs = []

        # Unroll LSTM through time
        # At each step, feed the word at position t and carry h and c forwar
        for t in range(L):
            h, c = self.lstm_cell(emb[:, t], h, c)
            hs.append(h.unsqueeze(1))  # (B, H) -> (B, 1, H) so we can conca

        Hseq = torch.cat(hs, dim=1)  # (B, L, H)

        # Extract last valid hidden state
        # lengths - 1 = index of the last real token; clamp_min(0) protects
        # (length 0) from becoming index -1. Advanced indexing then picks, f
        # example b, the row Hseq[b, idx[b]] -> result shape (B, H).
        idx = (lengths - 1).clamp_min(0)
        batch_indices = torch.arange(B, device=emb.device)
        last_h = Hseq[batch_indices, idx]
```

```
        return self.fc(last_h)

# Instantiate LSTM model
model_lstm = LSTMWithEmbedding(max_token_len, vocab_size, embedding_dim, hid
print(model_lstm)

# Train the model (uncomment after implementing the LSTM cell)
history_lstm = train(model_lstm, dataloader, dataloader_val, num_epochs=5, l
```

```
LSTMWithEmbedding(
  (embedding): Embedding(15994, 50, padding_idx=0)
  (lstm_cell): LSTMCell()
  (fc): Linear(in_features=100, out_features=4, bias=True)
)
100%|██████████| 375/375 [00:29<00:00, 12.85it/s]
100%|██████████| 238/238 [00:05<00:00, 43.23it/s]
Epoch [1/5] Train: 1.1369 Val: 0.9798 Acc: 59.75%
100%|██████████| 375/375 [00:28<00:00, 13.17it/s]
100%|██████████| 238/238 [00:05<00:00, 41.26it/s]
Epoch [2/5] Train: 0.6805 Val: 0.7261 Acc: 73.07%
100%|██████████| 375/375 [00:26<00:00, 14.08it/s]
100%|██████████| 238/238 [00:05<00:00, 44.76it/s]
Epoch [3/5] Train: 0.4765 Val: 0.6559 Acc: 76.21%
100%|██████████| 375/375 [00:27<00:00, 13.80it/s]
100%|██████████| 238/238 [00:05<00:00, 45.51it/s]
Epoch [4/5] Train: 0.3662 Val: 0.6171 Acc: 78.84%
100%|██████████| 375/375 [00:27<00:00, 13.62it/s]
100%|██████████| 238/238 [00:05<00:00, 47.17it/s]
Epoch [5/5] Train: 0.2811 Val: 0.6062 Acc: 78.43%
```

Part 5: LSTM Language Model (20 points)

Implement an LSTM-based language model that predicts the next token in a sequence. Unlike classification, this model outputs a probability distribution over the vocabulary for each token position.

In [12]:
```
# Language model: at EVERY position t, predict the next word (t+1).
# Difference from the classifier: instead of one prediction from the last hi
# state, we produce a score for every vocabulary word at every time step.
class LSTMLanguageModel(nn.Module):
    def __init__(self, vocab_size: int, embedding_dim: int, hidden_size: int
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx
        self.lstm_cell = LSTMCell(embedding_dim, hidden_size)

        # TODO: Create output layer for language modeling (3 points)
        # Hint: The output layer should map from hidden_size to vocab_size
        # This produces logits for each token in the vocabulary
        self.fc = nn.Linear(hidden_size, vocab_size)

        self.hidden_size = hidden_size
        self.vocab_size = vocab_size
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (B, L) token ids. No `lengths` here: padding positions are hand
        # ignore_index in the loss instead.
        # TODO: Get embeddings (2 points)
        # Hint: emb = self.embedding(x)
        emb = self.embedding(x)

        B, L, E = emb.size()

        # TODO: Initialize hidden and cell states (3 points)
        # Hint: Create two zero tensors of shape (B, self.hidden_size)
        h = torch.zeros(B, self.hidden_size, device=emb.device)
        c = torch.zeros(B, self.hidden_size, device=emb.device)

        outputs = []  # next-word logits for every time step

        # TODO: Unroll LSTM and collect outputs at each time step (7 points)
        # Hint 1: Loop through time: for t in range(L)
        # Hint 2: Update LSTM: h, c = self.lstm_cell(emb[:, t], h, c)
        # Hint 3: Get output logits: out = self.fc(h)
        # Hint 4: Store output: outputs.append(out.unsqueeze(1))
        #TODO: Implement time loop

        for t in range(L):
            h, c = self.lstm_cell(emb[:, t], h, c)
            out = self.fc(h)
            outputs.append(out.unsqueeze(1))

        # TODO: Concatenate outputs (2 points)
        # Hint: output_seq = torch.cat(outputs, dim=1)
        # This creates a tensor of shape (B, L, vocab_size)
        output_seq = torch.cat(outputs, dim=1)

        return output_seq
```

In [13]:

```
def train_language_model(model, dataloader, dataloader_val, num_epochs: int
    """Training function for language modeling."""
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

    # TODO: Define loss function for language modeling (3 points)
    # Hint 1: Use nn.CrossEntropyLoss()
    # Hint 2: Set ignore_index=tokenizer.pad_idx to ignore padding tokens
    # This is important so we don't penalize the model for predicting paddin
    criterion = nn.CrossEntropyLoss(ignore_index=tokenizer.pad_idx)

    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    history = {"train_loss": [], "val_loss": [], "perplexity": []}

    for epoch in range(num_epochs):
        model.train()
        total_loss = 0.0

        # Labels and lengths from the dataset are not used (hence `_`)
        for inputs, _, _ in tqdm(dataloader, desc=f"Epoch {epoch+1}/{num_epo
```

```
            inputs = inputs.to(device)

            # Create targets by shifting input by 1 position
            # Example: inputs  = [the, cat, sat, on, <pad>]
            #          targets = [cat, sat, on, <pad>, <pad>]
            # so at position t the model must predict word t+1. The last pos
            # 0 (= <pad> id), which is ignored by the loss.
            targets = torch.cat([inputs[:, 1:], torch.zeros(inputs.size(0),

            optimizer.zero_grad()
            outputs = model(inputs)  # (B, L, V)

            # Reshape for loss computation
            # CrossEntropyLoss expects (N, num_classes) and (N,), so flatten
            # time together: every (example, position) pair becomes one pred
            outputs_flat = outputs.view(-1, model.vocab_size)  # (B*L, V)
            targets_flat = targets.view(-1)                    # (B*L,)

            loss = criterion(outputs_flat, targets_flat)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        # Validation
        # Same computation without gradient updates, to measure generalizati
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for inputs, _, _ in tqdm(dataloader_val, desc=f"Epoch {epoch+1}/
                inputs = inputs.to(device)
                targets = torch.cat([inputs[:, 1:], torch.zeros(inputs.size(

                outputs = model(inputs)
                outputs_flat = outputs.view(-1, model.vocab_size)
                targets_flat = targets.view(-1)

                loss = criterion(outputs_flat, targets_flat)
                val_loss += loss.item()

        avg_train_loss = total_loss / len(dataloader)
        avg_val_loss = val_loss / len(dataloader_val)
        # Perplexity = exp(average cross-entropy). Intuition: the model is a
        # as if it were choosing uniformly among `perplexity` words. Lower i
        perplexity = torch.exp(torch.tensor(avg_val_loss)).item()

        history["train_loss"].append(avg_train_loss)
        history["val_loss"].append(avg_val_loss)
        history["perplexity"].append(perplexity)

        print(f"Epoch [{epoch+1}/{num_epochs}] | Train Loss: {avg_train_loss

    return history

# Create language modeling dataset
lm_dataset = LanguageModelDataset(data, tokenizer, max_length=max_token_len)
lm_dataset_val = LanguageModelDataset(data_val, tokenizer, max_length=max_to
```

```
lm_dataloader = torch.utils.data.DataLoader(lm_dataset, batch_size=32, shuff
lm_dataloader_val = torch.utils.data.DataLoader(lm_dataset_val, batch_size=3

x, _, _ = next(iter(lm_dataloader))
print(LSTMLanguageModel(vocab_size, embedding_dim, hidden_size, pad_idx)(x).

# Instantiate and train (uncomment after implementing the above TODOs)
language_model = LSTMLanguageModel(vocab_size, embedding_dim, hidden_size, p
history_lm = train_language_model(language_model, lm_dataloader, lm_dataload
```

```
torch.Size([32, 128, 15994])
Epoch 1/5 [Train]: 100%|██████████| 375/375 [04:42<00:00,  1.33it/s]
Epoch 1/5 [Val]: 100%|██████████| 238/238 [01:14<00:00,  3.20it/s]
Epoch [1/5] | Train Loss: 7.5806 | Val Loss: 7.0358 | Val Perplexity: 1136.58
Epoch 2/5 [Train]: 100%|██████████| 375/375 [05:08<00:00,  1.22it/s]
Epoch 2/5 [Val]: 100%|██████████| 238/238 [01:08<00:00,  3.49it/s]
Epoch [2/5] | Train Loss: 7.0325 | Val Loss: 6.7710 | Val Perplexity: 872.14
Epoch 3/5 [Train]: 100%|██████████| 375/375 [04:44<00:00,  1.32it/s]
Epoch 3/5 [Val]: 100%|██████████| 238/238 [01:08<00:00,  3.47it/s]
Epoch [3/5] | Train Loss: 6.7251 | Val Loss: 6.5886 | Val Perplexity: 726.77
Epoch 4/5 [Train]: 100%|██████████| 375/375 [04:50<00:00,  1.29it/s]
Epoch 4/5 [Val]: 100%|██████████| 238/238 [01:09<00:00,  3.40it/s]
Epoch [4/5] | Train Loss: 6.4717 | Val Loss: 6.4529 | Val Perplexity: 634.56
Epoch 5/5 [Train]: 100%|██████████| 375/375 [04:48<00:00,  1.30it/s]
Epoch 5/5 [Val]: 100%|██████████| 238/238 [01:08<00:00,  3.47it/s]
Epoch [5/5] | Train Loss: 6.2566 | Val Loss: 6.3540 | Val Perplexity: 574.76
```

Bonus: Stabilize the Stacked RNN

If you trained the 3-layer stacked RNN in Part 3, you may have noticed that its training is unstable: the validation accuracy jumps around between epochs and can drop sharply in the last epochs, sometimes ending up worse than the single-layer RNN from Part 2. This happens even with a correct implementation.

Your task is to figure out why, and fix it.

1. **Diagnose** find out what goes wrong during training. Support your answer with evidence from your own experiments (something you measured or printed while training), not just a guess.
2. **Fix** change *how the model is trained* so that the 3-layer stacked RNN trains reliably. Rules:
   - Keep the Part 3 model as it is: `RNNWithStackedLayers` with `num_layers=3` and the same `embedding_dim`, `hidden_size`, data and 5 epochs.
   - Do not edit the provided `train()` function above (Parts 2-4 rely on it). Write your own training function in the cell below instead.
   - Show the per-epoch validation accuracy of the original and your fixed training side by side. A single run can be lucky, so compare more than one random

seed (`torch.manual_seed(...)` before creating each model).

3. **Explain (1 point):** in a few sentences, explain why your fix works.

In [14]:
```
# Bonus: your code here (diagnosis experiments, your training function, and
```

Bonus answer (diagnosis and explanation):

Your answer here

Submission Instructions

Before Submitting

1. Make sure you have implemented all TODO sections
2. Uncomment the training code and run all cells
3. Verify that your models train successfully
4. Check that you don't have any NotImplementedError exceptions

What to Submit

- This completed Jupyter notebook (.ipynb file)
- A PDF export of the notebook with all outputs visible

Grading Rubric

Part	Description	Points
1	RNN Cell forward pass implementation	20
2	RNN with Embedding forward pass	15
3	Stacked RNN implementation	20
4	LSTM Cell implementation	25
5	Language Model implementation	20
Total		**100**

Tips for Success

- Read all hints carefully - they guide you through the implementation
- Pay attention to tensor shapes mentioned in comments
- Test each part incrementally before moving to the next
- Compare your validation accuracy with expected ranges:
  - RNN: ~60-70% accuracy on AG News
  - LSTM: ~65-75% accuracy on AG News
- Use `print()` statements to debug tensor shapes if needed

- Runtime: the classifiers (Parts 2-4) take about 1 minute each for 5 epochs on a recent laptop CPU. The language model (Part 5) is much heavier: about 7 minutes on an Apple M4 CPU and considerably longer on older laptops. If it is too slow, run the notebook on Google Colab with a GPU (Runtime > Change runtime type > GPU).

Good luck!